

МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ

ХАРКІВСЬКИЙ НАЦІОНАЛЬНИЙ
УНІВЕРСИТЕТ РАДІОЕЛЕКТРОНІКИ

Кафедра «Програмна інженерія»

ЗВІТ

з лабораторної роботи №2

з дисципліни «Аналіз та рефакторинг коду»

на тему «Розробка бази даних для серверної частини програмної системи
та прикладного програмного інтерфейсу (API)»

Виконав:

ст. гр. ПЗПІ-23-8

Потурайко Н.І.

Прийняв:

асистент. каф. ПІ.

Дашенков Д. С.

Харків 2025

2.1 Мета роботи

Метою лабораторної роботи є розробка серверної частини програмної системи Aquila для автоматизації спостереження за кімнатою, що включає проєктування реляційної бази даних для зберігання показників мультисенсорних модулів та журналів подій, побудову необхідних UML та ER діаграм, а також програмну реалізацію REST API для забезпечення надійного обміну даними між IoT-пристроями контролю середовища, веб-інтерфейсом користувача та сервером.

2.2 UML діаграма прецедентів

Для візуалізації функціональних вимог до серверної частини системи Aquila та визначення меж проєкту було розроблено діаграму прецедентів. Дана діаграма відображає основні варіанти використання системи та взаємодію зовнішніх акторів із функціоналом бекенду. Основними акторами системи визначено Користувача, Адміністратора та IoT-пристрій. Користувач взаємодіє із системою через клієнтський інтерфейс, маючи можливість проходити автентифікацію, переглядати поточні показники мікроклімату кімнати, аналізувати історію змін температури, вологості та рівня шуму, а також отримувати сповіщення про виявлення руху або перевищення порогових значень. Адміністратор має розширені повноваження, що включають управління списком зареєстрованих пристроїв та конфігурацію системних параметрів. IoT-пристрій виступає як автономний актор, що ініціює передачу зібраних телеметричних даних на сервер та періодично запитує оновлення конфігурації. Логіка обробки всіх наведених прецедентів реалізується на стороні серверної частини через відповідні API-ендпоінти.

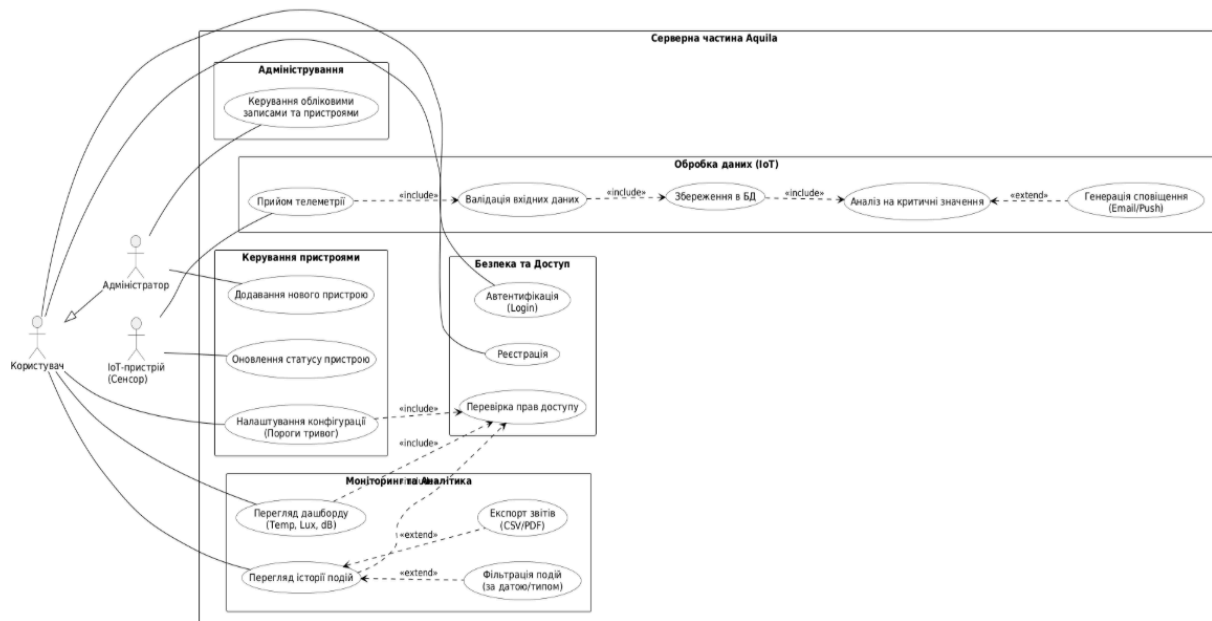


Рисунок 2.1 – UML діаграма прецедентів

2.3 ER діаграма

Для реалізації розширеного функціонала системи Aquila, що включає логічне зонування приміщень та візуальний контроль подій, було розроблено структуру бази даних. На верхньому рівні ієрархії знаходиться сутність User, яка пов'язана із сутністю Room зв'язком «один-до-багатьох». Це дозволяє користувачу створювати віртуальну модель свого приміщення (наприклад, «Вітальня», «Серверна», «Коридор») для зручнішого групування техніки.

Сутність Device тепер підпорядкована конкретній кімнаті, що забезпечує чітку локалізацію джерел даних. Окрім стандартної сутності Measurement (для числових часових рядів: температура, вологість, шум), введено нову сутність Snapshot. Вона призначена для зберігання метаданих фотознімків, зроблених камерами спостереження при спрацюванні тригерів безпеки. Таблиця зберігає посилання на файл зображення (URL), час фіксації та причину спрацювання (наприклад, «Motion Detected»). Така схема забезпечує гнучкість масштабування системи від однієї кімнати до

комплексу приміщень та надає можливість мультимодального моніторингу (числові дані + візуальні докази).

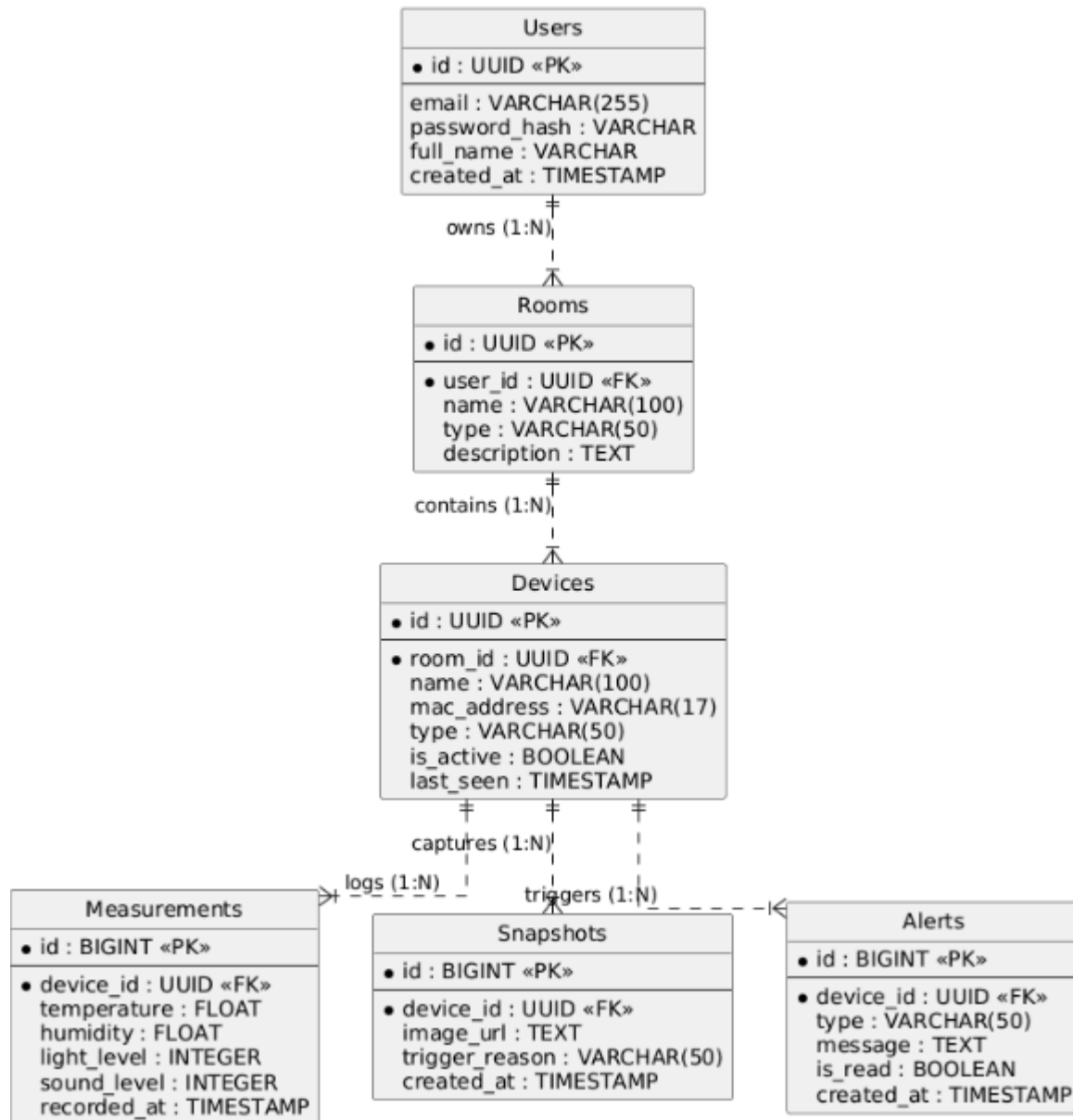


Рисунок 2.2 – ER діаграма

2.4 Структура бази даних.

Фізична реалізація бази даних системи Aquila виконана у СУБД PostgreSQL. Вибір цієї системи зумовлений її надійністю, підтримкою

складних зв'язків, ACID-транзакцій та можливістю роботи з JSON-даними, що є важливим для гнучкості IoT-конфігурацій.

Для забезпечення масштабованості та безпеки як первинні ключі для основних сутностей (Users, Rooms, Devices) використано UUID (Universally Unique Identifier). Це дозволяє уникнути конфліктів при розподіленому створенні записів та ускладнює перебір ідентифікаторів злоумисниками. Для таблиць з великою кількістю записів (Measurements, Snapshots, Alerts) використано тип BIGINT для оптимізації індексації та економії місця.

```
CREATE TABLE users (  
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
    email VARCHAR(255) UNIQUE NOT NULL,  
    password_hash VARCHAR NOT NULL,  
    full_name VARCHAR(100),  
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP  
);
```

```
CREATE TABLE rooms (  
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
    user_id UUID NOT NULL REFERENCES users(id) ON DELETE CASCADE,  
    name VARCHAR(100) NOT NULL,  
    type VARCHAR(50),  
    description TEXT,  
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP  
);
```

```
CREATE TABLE devices (  
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
    room_id UUID REFERENCES rooms(id) ON DELETE SET NULL,  
    name VARCHAR(100) NOT NULL,  
    mac_address VARCHAR(17) UNIQUE NOT NULL,  
    type VARCHAR(50) NOT NULL,  
    is_active BOOLEAN DEFAULT TRUE,
```

```
        last_seen TIMESTAMP,  
        created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP  
    );
```

```
CREATE TABLE measurements (  
    id BIGSERIAL PRIMARY KEY,  
    device_id UUID NOT NULL REFERENCES devices(id) ON DELETE  
CASCADE,  
    temperature FLOAT,  
    humidity FLOAT,  
    light_level INTEGER,  
    sound_level INTEGER,  
    recorded_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP  
);
```

```
CREATE TABLE snapshots (  
    id BIGSERIAL PRIMARY KEY,  
    device_id UUID NOT NULL REFERENCES devices(id) ON DELETE  
CASCADE,  
    image_url TEXT NOT NULL,  
    trigger_reason VARCHAR(50),  
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP  
);
```

```
CREATE TABLE alerts (  
    id BIGSERIAL PRIMARY KEY,  
    device_id UUID NOT NULL REFERENCES devices(id) ON DELETE  
CASCADE,  
    type VARCHAR(50) NOT NULL,  
    message TEXT NOT NULL,  
    is_read BOOLEAN DEFAULT FALSE,  
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP  
);
```

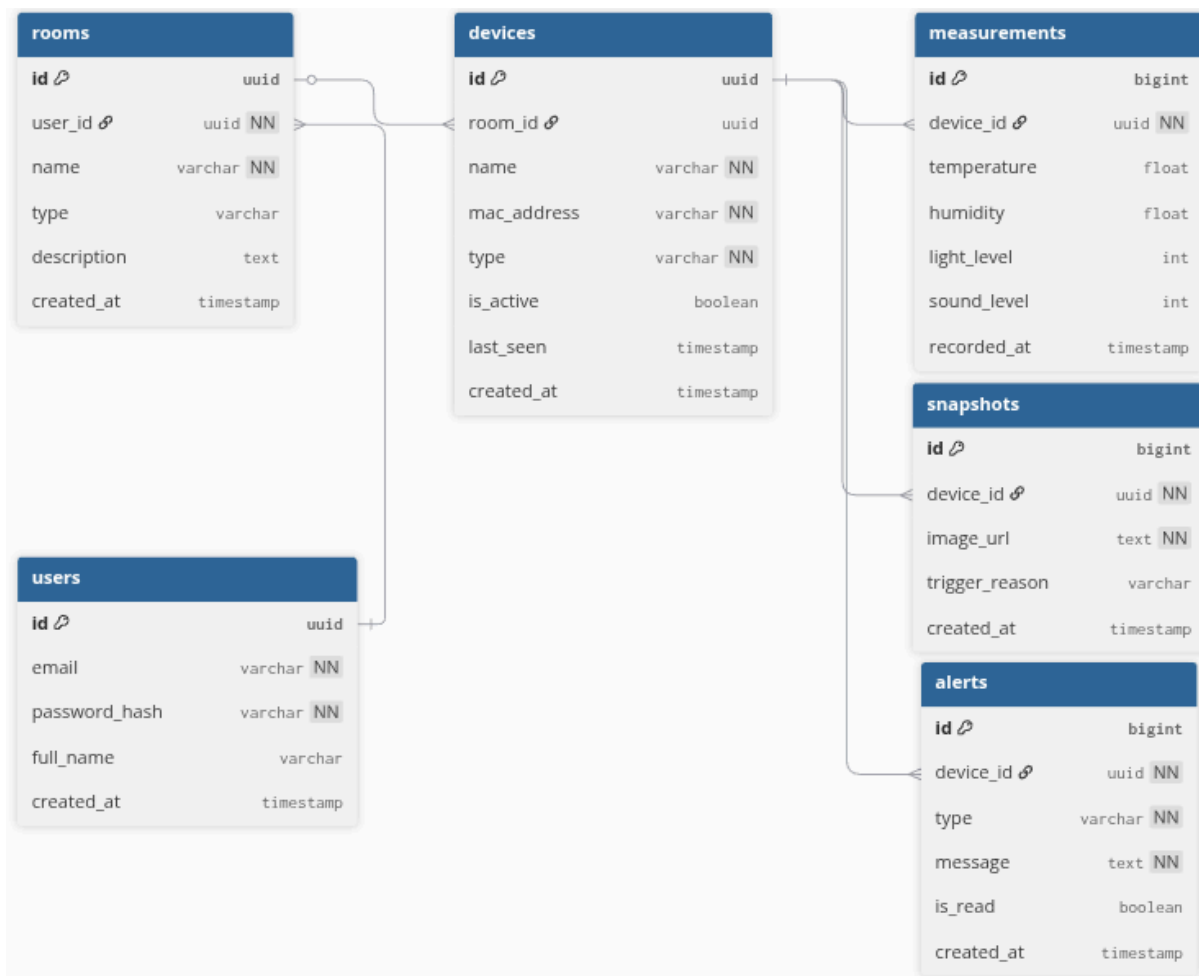


Рисунок 2.3 – Зображення бази даних

2.5 Розробка API

Для забезпечення взаємодії між клієнтською частиною, IoT-пристроями та сервером було розроблено RESTful API на базі платформи Node.js та фреймворку Express. Архітектура API орієнтована на ресурси (Resource-oriented), де основними ресурсами виступають кімнати (rooms), пристрої (devices) та телеметричні дані (telemetry).

Обмін даними здійснюється у форматі JSON. Для кожного запиту реалізовано валідацію вхідних даних на відповідність типам та обов'язковим полям. Нижче наведено специфікацію основних розроблених ендпоінтів (маршрутів) системи Aquila.

Для забезпечення взаємодії між клієнтською частиною, IoT-пристроями та сервером було розроблено RESTful API на базі платформи Node.js та фреймворку Express. Архітектура API орієнтована на ресурси (Resource-oriented), де основними ресурсами виступають кімнати (rooms), пристрої (devices) та телеметричні дані (telemetry).

Обмін даними здійснюється у форматі JSON. Для кожного запиту реалізовано валідацію вхідних даних на відповідність типам та обов'язковим полям.

1. Група маршрутів: Керування кімнатами (Zoning)

Ці маршрути дозволяють створювати логічні зони (кімнати) та отримувати їх список.

Маршрут: GET /api/rooms

Призначення: Отримання списку всіх кімнат користувача з переліком підключених до них пристроїв.

Відповідь: Масив об'єктів Room, кожен з яких містить вкладений масив devices.

Маршрут: POST /api/rooms

Призначення: Створення нової кімнати (зони).

Вхідні дані (Body): { "userId": "uuid", "name": "Kitchen", "type": "General" }.

Логіка: Створює запис у таблиці rooms, повертає ID створеної кімнати.

2. Група маршрутів: Реєстрація пристроїв

Забезпечує прив'язку фізичних IoT-пристроїв до конкретних кімнат.

Маршрут: POST /api/devices

Призначення: Реєстрація нового сенсора або камери в системі.

Вхідні дані (Body): { "roomId": "uuid", "name": "Main Sensor", "macAddress": "AA:BB:CC:11:22", "type": "Multisensor" }.

Валідація: Перевіряє, чи не зареєстрована вже така MAC-адреса. Якщо ні — створює запис і прив'язує до кімнати.

3. Група маршрутів: Прийом телеметрії (IoT Data Ingestion)

Критично важлива частина API, призначена для високочастотного прийому даних від мікроконтролерів (ESP32).

Маршрут: POST /api/telemetry

Призначення: Збереження показників мікроклімату.у

Алгоритм роботи:

Сервер шукає пристрій за macAddress.

Якщо пристрій знайдено, оновлює поле last_seen.

Записує нові дані у таблицю measurements.

Перевіряє показники на критичні рівні для генерації тривоги.

4. Група маршрутів: Фотофіксація подій

Маршрут: POST /api/snapshots

Призначення: Збереження посилання на фото, зроблене камерою при виявленні руху.

Вхідні дані (Body): { "macAddress": "...", "imageUrl": "http://cloud/img1.jpg", "reason": "Motion Detected" }.

Логіка: Фіксує подію у таблиці snapshots з прив'язкою до часу та пристрою.

2.6 Висновки

У ході виконання лабораторної роботи було спроектовано та реалізовано северну частину програмної системи Aquila для автоматизації спостереження за приміщенням. Розроблено діаграму прецедентів, яка визначила ролі користувачів та сценарії використання системи. Створено ER-діаграму, що дозволило реалізувати розширену модель даних із підтримкою зонування приміщень (сутність Rooms) та фотофіксації подій (сутність Snapshots). Розгорнуто базу даних PostgreSQL, створено таблиці з використанням UUID для первинних ключів та налаштовано відповідні індекси для оптимізації запитів. Реалізовано REST API на Node.js, що забезпечує реєстрацію пристроїв, створення кімнат та прийом телеметричних даних. Окрему увагу приділено структурі запитів від

IoT-пристроїв, що дозволяє легко інтегрувати апаратну частину із сервером.